YOLO — **computer vision’da real-time object detection uchun eng mashhur model oilalaridan biri**. YOLO nomi **“You Only Look Once”** degan iboradan olingan. Ma’nosi: model rasmga “bir marta qarab”, undagi obyektlar qayerda joylashganini va qaysi classga tegishli ekanini birdaniga taxmin qiladi. Original YOLO paper’da detection vazifasi “rasm pixellaridan to‘g‘ridan-to‘g‘ri bounding box koordinatalari va class probability’larni bashorat qiladigan bitta regression problem” sifatida ta’riflangan.

YOLO’ni **nazariya, arxitektura, dataset, training, evaluation, inference, deployment, xatolar va amaliy kodlar** bilan batafsil tushunish

YOLO so‘zining to‘liq shakli:

```
You Only Look Once
```

O‘zbekchasi:

```
Siz faqat bir marta qaraysiz
```

Lekin computer vision’da ma’nosi oddiy tarjimadan ko‘ra muhimroq:

YOLO rasmga bir marta qarab, shu rasm ichida qanday obyektlar borligini va ular qayerda joylashganini topadi.

Masalan, rasmda odam, mashina va it bo‘lsa, YOLO bitta ishlash jarayonida shularni chiqaradi:

```
person  → qayerda turgani + ishonch foizi
car     → qayerda turgani + ishonch foizi
dog     → qayerda turgani + ishonch foizi
```

Ya’ni YOLO shuni qiladi:

```
Rasm → model → obyekt nomi + obyekt joyi + confidence
```

Masalan:

```
person: 0.92
car: 0.87
dog: 0.76
```

Bu yerda `0.92`, `0.87`, `0.76` — modelning ishonch darajasi.

Nega “You Only Look Once” deyilgan?

Chunki eski object detection usullarida model rasmning turli joylariga qayta-qayta qarab, obyekt qidirardi. YOLO esa rasmni bitta neural network orqali bir marta o‘tkazadi va birdaniga obyektlarni topadi.

Oddiy qilib:

```
Eski usul:
rasmning har joyiga alohida-alohida qaraydi

YOLO:
butun rasmga bir marta qaraydi va hammasini birdan topadi
```

Shuning uchun YOLO juda tez ishlaydi. Ayniqsa video, kamera, CCTV, robot, avtomobil, drone kabi real-time vazifalarda ko‘p ishlatiladi.

Eng sodda xulosa:

YOLO = You Only Look Once = rasmga bir marta qarab obyektlarni topadigan model.

U uchta asosiy natija beradi:

```
1. Nima bor?        → class, masalan person, car, dog
2. Qayerda bor?   → bounding box
3. Qanchalik ishonchli? → confidence score
```

1. YOLO nima?

YOLO — rasm yoki videodagi obyektlarni topadigan deep learning modeli.

Masalan, modelga mana shunday rasm kiradi:

```
image.jpg
```

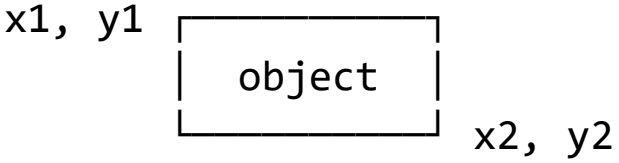
Model natijasi shunaqa bo‘ladi:

```
person   confidence=0.92   box=(x1, y1, x2, y2)
car      confidence=0.87   box=(x1, y1, x2, y2)
dog      confidence=0.76   box=(x1, y1, x2, y2)
```

Ya’ni YOLO uchta asosiy narsani chiqaradi:

Natija	Ma’nosi
Class	Obyekt turi: person, car, dog, helmet, apple
Bounding box	Obyekt rasmning qayerida turgani
Confidence	Modelning ishonch darajasi

Bounding box odatda obyekt atrofidagi to‘rtburchak:



YOLO ayniqsa **real-time** ishlash uchun mashhur. Original YOLO ishida model rasmni bitta network evaluation orqali ishlagani sababli tezlik asosiy kuchli tomonlardan biri bo‘lgan.

2. YOLO faqat object detection emas

Avval YOLO asosan object detection uchun tanilgan edi. Lekin hozirgi Ultralytics YOLO ekotizimida bir nechta computer vision task bor. Official Ultralytics hujjatlarida YOLO oilasi detection, instance segmentation, semantic segmentation, classification, pose estimation, tracking va oriented object detection kabi vazifalarni qo‘llashi ko‘rsatilgan.

YOLO bilan bajariladigan asosiy vazifalar:

Task	Ma’nosi	Misol
Detection	Obyektni box bilan topish	odam, mashina, daraxt
Instance segmentation	Har bir obyektni mask bilan ajratish	har bir odam alohida mask
Semantic segmentation	Har bir pixelga class berish	road, sky, car
Classification	Butun rasmga bitta class berish	cat/dog
Pose estimation	Keypointlarni topish	odam skeleti, qo‘l, tizza
Tracking	Videoda obyektni kuzatish	person ID 1, car ID 2
OBB	Aylantirilgan box bilan detection	ship, text, aerial image

3. YOLO nima uchun mashhur?

YOLO’ning mashhurligi bir nechta sababga ega:

1. **Tez ishlaydi.** Video, webcam, CCTV, robot, drone kabi real-time vazifalarda ishlatish qulay.
2. **Training qilish nisbatan oson.** Ayniqsa Ultralytics paketi bilan custom datasetda train qilish juda sodda.

3. **Pretrained model bor.** COCO kabi katta datasetlarda oldindan o‘qitilgan modelni olib, o‘z datasetingizga fine-tune qilish mumkin.
4. **Deployment qulay.** ONNX, TensorRT, CoreML, OpenVINO kabi formatlarga export qilish mumkin. Ultralytics export hujjatlarida YOLO modelini turli platforma va qurilmalar uchun export qilish imkoniyatlari berilgan.
5. **Bir framework ichida ko‘p task bor.** Detection, segmentation, pose, classification, tracking bitta API bilan ishlaydi.

4. Object detection nima?

YOLO’ni tushunish uchun avval object detection’ni tushunish kerak.

Image classification faqat bitta savolga javob beradi:

```
Bu rasmda nima bor?
```

Masalan:

```
dog
```

Object detection esa ikki savolga javob beradi:

```
Nima bor?
Qayerda bor?
```

Masalan:

```
dog  → box koordinatalari
person → box koordinatalari
car → box koordinatalari
```

Shuning uchun object detection classification’dan murakkabroq. Chunki model faqat classni emas, joylashuvni ham topishi kerak.

5. YOLO qanday ishlaydi?

YOLO’ni oddiy pipeline sifatida shunday tasavvur qilish mumkin:

```
Input image
   ↓
Resize / preprocessing
   ↓
Backbone
   ↓
Neck
   ↓
Head
   ↓
Prediction
   ↓
NMS yoki end-to-end filtering
   ↓
Final boxes/classes/confidence
```

Endi har bir qismini tushuntiraman.

6. Input image va resize

YOLO modelga rasm kiradi. Ko‘pincha rasm `640×640` size’ga keltiriladi:

```
original image → resized image 640×640
```

Nega?

Chunki deep learning model fixed yoki ma’lum tartibdagi tensor size bilan ishlaydi. YOLO training va inference paytida `imgsz=640` kabi parametr ko‘p ishlatiladi. Ultralytics misollarida ham train qilishda `imgsz=640` ishlatilgan.

Lekin bu har doim 640 bo‘lishi shart emas. Kichik obyektlar ko‘p bo‘lsa, `imgsz=960` yoki `1280` yordam berishi mumkin. Lekin image size oshsa, GPU memory va inference vaqti ham oshadi.

7. Backbone nima?

Backbone — rasm ichidan feature chiqaradigan asosiy qism.

Oddiy qilib:

```
image → backbone → feature maps
```

Backbone rasm ichidan quyidagilarni o‘rganadi:

- edge,
- corner,
- texture,
- shape,
- object parts,
- murakkab visual patternlar.

Masalan, odamni aniqlash uchun model bosh, tana, qo‘l, oyoq, kiyim, rang, kontur kabi patternlarni bilishi kerak.

YOLO’da backbone rasmni birdan “person” deb aytmaydi. U avval rasmni feature maplarga aylantiradi.

8. Neck nima?

Neck — backbone’dan kelgan feature’larni birlashtiradigan qism.

Nega kerak?

Rasmda obyektlar har xil o‘lchamda bo‘ladi:

```
katta obyekt: mashina yaqin turgan
o‘rta obyekt: odam
kichik obyekt: uzoqdagi qush yoki helmet
```

Backbone turli leveldagi feature maplar chiqaradi. Neck ularni birlashtirib, modelga kichik va katta obyektlarni yaxshiroq topishga yordam beradi.

Ko‘p YOLO variantlarida neck multi-scale feature fusion uchun ishlatiladi. Amaliy ma’noda neck modelga shunday yordam beradi:

```
kichik obyektlarni ham ko‘rish
katta obyektlarni ham to‘g‘ri tushunish
contextni birlashtirish
```

9. Head nima?

Head — oxirgi prediction chiqaradigan qism.

Detection head odatda shularni taxmin qiladi:

```
box coordinates
class scores
confidence / objectness yoki class confidence
```

Model natijasi ko‘p candidate boxlar bo‘lishi mumkin. Masalan, model bitta odam uchun bir nechta box chiqarishi mumkin. Keyin ortiqcha boxlarni kamaytirish kerak bo‘ladi.

10. Bounding box formatlari

Bounding box bir nechta formatda ifodalanishi mumkin.

10.1. `x1, y1, x2, y2`

Bu formatda boxning chap-yuqori va o‘ng-past koordinatalari beriladi:

```
x1 = chap tomon
y1 = yuqori tomon
x2 = o‘ng tomon
y2 = past tomon
```

10.2. `x_center, y_center, width, height`

YOLO label formatida ko‘pincha shu ishlatiladi:

```
class_id x_center y_center width height
```

Masalan:

```
0 0.512 0.438 0.230 0.360
```

Bu degani:

```
class_id = 0
x_center = 0.512
y_center = 0.438
```

```
width = 0.230
height = 0.360
```

Muhim: YOLO label formatida koordinatalar odatda **normalized** bo‘ladi, ya’ni 0 dan 1 gacha.

```
0.5 = rasmning 50% joyi
1.0 = to‘liq rasm o‘lchami
```

Ultralytics detection dataset formatida image va label papkalari hamda YOLO formatdagi normalized koordinatalar ishlatiladi.

11. Confidence nima?

Confidence — modelning obyekt borligiga va class to‘g‘riligiga ishonchi.

Masalan:

```
person 0.94
car    0.81
dog    0.42
```

Inference paytida siz `conf` threshold qo‘yasiz:

```
conf=0.25
```

Bu degani:

```
confidence 0.25 dan past predictionlar tashlab yuboriladi
```

Agar `conf` juda past bo‘lsa:

```
ko‘p false positive chiqadi
```

Agar `conf` juda yuqori bo‘lsa:

```
ba’zi haqiqiy obyektlar topilmay qoladi
```

Shuning uchun threshold tanlash vazifaga bog‘liq.

Masalan, xavfsizlik helmet detection’da no_helmetni o‘tkazib yubormaslik muhim bo‘lsa, recall muhim. Bunda thresholdni juda baland qo‘yish xavfli.

12. IoU nima?

IoU — Intersection over Union.

Bu predicted box bilan true box qanchalik ustma-ust tushganini o‘lchaydi.

Formula:

```
IoU = intersection area / union area
```

Misol:

```
IoU = 0.90 → juda yaxshi box
IoU = 0.50 → o‘rtacha
IoU = 0.10 → yomon
```

Object detection evaluation’da IoU juda muhim. Chunki model classni to‘g‘ri topsa ham, box noto‘g‘ri bo‘lsa, natija yomon hisoblanadi.

13. NMS nima?

NMS — Non-Maximum Suppression.

Model bitta obyekt uchun bir nechta box chiqarishi mumkin:

```
person box 1: 0.92
person box 2: 0.87
person box 3: 0.74
```

NMS eng yaxshi boxni qoldirib, juda o‘xshash boxlarni olib tashlaydi.

Oddiy g‘oya:

```
1. Eng yuqori confidence box olinadi.
2. Unga juda o‘xshash boxlar topiladi.
3. IoU yuqori bo‘lsa, ular olib tashlanadi.
4. Keyingi boxga o‘tiladi.
```

Hozirgi yangi YOLO yo‘nalishlarida NMS’ni kamaytirish yoki butunlay default inference yo‘lidan olib tashlashga urinishlar bor. Ultralytics YOLO26 hujjatlarida end-to-end NMS-free inference, DFL-free regression va deployment uchun yengilroq head kabi yangiliklar sanab o‘tilgan.

14. YOLO versiyalari haqida umumiy tushuncha

YOLO bitta model emas, balki yillar davomida rivojlangan model oilasi.

Soddalashtirilgan tarix:

Versiya / oilalar	Umumiy g‘oya

YOLOv1	Detection’ni bitta regression problem sifatida berdi
YOLOv2 / YOLO9000	Anchor box, batch norm, yaxshilangan training
YOLOv3	Multi-scale detection, darknet backbone
YOLOv4	CSP, PAN, kuchli augmentationlar
YOLOv5	Ultralytics ekotizimida amaliy, keng ishlatilgan
YOLOv6/v7	Real-time detection bo‘yicha optimizatsiyalar
YOLOv8	Ultralytics’da anchor-free, tasklar kengaydi
YOLOv9/v10	PGI, NMS-free/efficiency yo‘nalishlari
YOLO11	Ultralytics’da detection, segmentation, pose, classification, OBB uchun keng model oilasi
YOLO12/YOLO26	attention, end-to-end/NMS-free va deploymentga moslashgan yangi yo‘nalishlar

Ultralytics model ro‘yxatida YOLOv9, YOLOv10 va YOLO11 alohida ko‘rsatilgan; YOLOv10 NMS-free training va efficiency-accuracy driven architecture bilan bog‘langan, YOLO11 esa detection, segmentation, pose, tracking va classification kabi tasklarda ishlatilishi aytilgan.

Hozir amaliyotda ko‘p foydalanuvchilar hali ham **YOLOv8** va **YOLO11** bilan ishlaydi, chunki ular keng dokumentatsiya, ko‘p tutorial va barqaror community tajribasiga ega. Ultralytics hujjatlarida YOLO11 model file nomlari `yolo11n.pt`, `yolo11s.pt`, `yolo11m.pt`, `yolo11l.pt`, `yolo11x.pt` ko‘rinishida berilgan va detection, segmentation, pose, OBB, classification variantlari borligi ko‘rsatilgan.

15. YOLO model size: n, s, m, l, x

YOLO’da model size ko‘pincha shunday bo‘ladi:

```
n = nano
s = small
m = medium
```

```
l = large
x = extra large
```

Masalan:

```
yolo11n.pt
yolo11s.pt
yolo11m.pt
yolo11l.pt
yolo11x.pt
```

Yoki yangi Ultralytics model oilalarida:

```
yolo26n.pt
yolo26s.pt
yolo26m.pt
yolo26l.pt
yolo26x.pt
```

Umumiy qoida:

Model	Tezlik	Accuracy	GPU talabi
n	eng tez	pastroq	kam
s	tez	yaxshi	kam/o‘rta
m	balans	yaxshi	o‘rta
l	sekinroq	yuqori	ko‘proq
x	eng og‘ir	eng yuqori bo‘lishi mumkin	katta

Boshlash uchun ko‘pincha `n` yoki `s` yaxshi:

```
model = YOLO("yolo11n.pt")
```

Agar dataset qiyin, obyektlar kichik, accuracy muhim bo‘lsa, keyin `m`, `l`, `x` sinab ko‘riladi.

16. YOLO qachon yaxshi tanlov?

YOLO ayniqsa quyidagi holatlarda juda yaxshi:

16.1. Real-time detection kerak bo‘lsa

Masalan:

```
webcam
CCTV
traffic camera
drone
robot
factory line camera
```

YOLO tezlik va accuracy balansini yaxshi beradi.

16.2. Custom object detection kerak bo‘lsa

Masalan:

```
helmet / no helmet
mask / no mask
car plate
fruit detection
plant disease spot
crack detection
product counting
```

Ultralytics API bilan custom training nisbatan oson.

16.3. Deployment kerak bo‘lsa

YOLO ONNX, TensorRT, CoreML, OpenVINO kabi formatlarga chiqarilishi mumkin. Ultralytics export hujjatlari real-world

deployment uchun turli formatlarga export qilishni asosiy maqsad sifatida beradi.

16.4. Detection bilan birga segmentation yoki pose ham kerak bo‘lsa

Ultralytics YOLO tasklar hujjatida detection, segmentation, semantic segmentation, classification, OBB va pose estimation bitta framework ichida ishlatilishi ko‘rsatilgan.

17. YOLO qachon yomon tanlov bo‘lishi mumkin?

YOLO kuchli, lekin har doim ham eng yaxshi tanlov emas.

17.1. Juda mayda obyektlar

Masalan:

```
uzoqdagi kichik qush
mikroskopdagi juda mayda hujayra
satellite image’da juda kichik obyekt
```

YOLO ishlashi mumkin, lekin image size, annotation sifati, tile/slicing strategy juda muhim bo‘ladi.

17.2. Juda yuqori aniqlikdagi segmentation kerak bo‘lsa

Agar pixel-level perfect mask kerak bo‘lsa, SAM, U-Net, Mask2Former, DeepLab, SegFormer kabi modellar bilan solishtirish kerak.

17.3. Open-vocabulary detection kerak bo‘lsa

Oddiy YOLO faqat o‘qitilgan classlarni topadi. Matn bilan “men xohlagan obyektni top” desangiz, CLIP/YOLO-World/YOLOE kabi open-vocabulary yo‘nalishlar kerak bo‘ladi. Ultralytics YOLOE hujjatlarida text, visual va internal vocabulary prompting qo‘llanishi aytilgan.

17.4. Juda kam data

YOLO pretrained modeldan boshlasa ham, object detection uchun annotation sifati juda muhim. Har bir obyektga box chizish kerak. 20–30 rasm bilan yaxshi generalization kutish qiyin.

18. YOLO dataset tuzilishi

Detection dataset odatda shunday bo‘ladi:

```
dataset/
  images/
    train/
      img001.jpg
      img002.jpg
    val/
      img101.jpg
      img102.jpg
    test/
      img201.jpg

  labels/
    train/
      img001.txt
```

```
      img002.txt
    val/
      img101.txt
      img102.txt
    test/
      img201.txt

  data.yaml
```

Har bir rasmga mos `.txt` label fayl bo‘ladi.

Masalan:

```
images/train/img001.jpg
labels/train/img001.txt
```

Agar `img001.jpg` ichida 2 ta obyekt bo‘lsa, `img001.txt` ichida 2 qator bo‘ladi:

```
0 0.512 0.438 0.230 0.360
1 0.230 0.614 0.120 0.210
```

Har bir qator:

```
class_id x_center y_center width height
```

Ultralytics detection dataset hujjatida YOLO formatda training/validation images va labels alohida papkalarda tashkil qilinishi ko‘rsatiladi.

19. `data.yaml` nima?

`data.yaml` — YOLO’ga dataset qayerda, classlar nechta va class nomlari qanday ekanini aytadigan config fayl.

Masalan:

```
path: data/helmet_dataset
train: images/train
val: images/val
test: images/test

names:
  0: helmet
  1: no_helmet
```

Yoki:

```
train: C:/Users/user/project/data/images/train
val: C:/Users/user/project/data/images/val

nc: 2
names: ["helmet", "no_helmet"]
```

`names` tartibi label fayldagi `class_id` bilan mos bo‘lishi shart.

Agar labelda:

```
0 0.5 0.5 0.2 0.3
```

bo‘lsa, `0` qaysi class ekanini `data.yaml` aytadi.

20. Annotation nima?

YOLO custom training uchun rasmda obyektlarga box chizish kerak.

Annotation tool’lar:

```
LabelImg
Roboflow
```

```
CVAT
Label Studio
makesense.ai
Ultralytics Platform
```

Har bir obyektga box chiziladi va class beriladi:

```
helmet
no_helmet
person
car
defect
```

Detection model sifati annotation sifatiga juda bog‘liq. Yomon box — yomon model.

21. Annotationda eng ko‘p xatolar

21.1. Box obyektni to‘liq qamramagan

Box juda kichik bo‘lsa, model obyekt chegarasini yomon o‘rganadi.

21.2. Box juda katta

Box ichiga fon ko‘p kirsa, model fonni ham obyekt deb o‘rganishi mumkin.

21.3. Class noto‘g‘ri qo‘yilgan

Masalan:

```
helmet o‘rniga no_helmet
```

Bu modelni chalkashtiradi.

21.4. Ba’zi obyektlar label qilinmagan

Rasmda 5 ta odam bor, lekin 3 tasi label qilingan. Model qolgan 2 tasini “background” deb o‘rganishi mumkin.

21.5. Train va val aralashib ketgan

Bir xil video frame’lari train va valga tushsa, validation score yolg‘on yuqori bo‘ladi.

22. YOLO training nima?

Training — modelga sizning rasmlaringiz va annotationlaringizni berib, obyektlarni o‘rganishga majbur qilish.

Oddiy training kodi:

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

results = model.train(
    data="data.yaml",
    epochs=100,
    imgsz=640,
    batch=16
)
```

Ultralytics YOLO11 hujjatlarida COCO pretrained `yolo11n.pt` modelni yuklab, `model.train(data="coco8.yaml", epochs=100, imgsz=640)` ko‘rinishida training qilish misoli berilgan.

23. Koddagi parametrlar ma’nosi

```
model = YOLO("yolo11n.pt")
```

Bu pretrained model yuklaydi.

```
data="data.yaml"
```

Dataset config fayl.

```
epochs=100
```

Datasetni 100 marta ko‘rib chiqadi.

```
imgsz=640
```

Training image size.

```
batch=16
```

Bir training step’da nechta image ishlatiladi.

24. Epoch nima?

Epoch — model butun training datasetni bir marta ko‘rib chiqishi.

Agar sizda 1000 ta rasm bo‘lsa:

```
1 epoch = 1000 rasmni bir marta ko‘rish
100 epoch = datasetni 100 marta ko‘rish
```

Lekin epoch ko‘p bo‘lsa har doim yaxshi emas. Dataset kichik bo‘lsa, model overfit bo‘lishi mumkin.

25. Batch size nima?

Batch size — bir vaqtda modelga nechta rasm berilishi.

Masalan:

```
batch=16
```

Bu degani model har step’da 16 ta rasm bilan update qiladi.

Katta batch:

```
tezroq bo‘lishi mumkin
GPU memory ko‘p talab qiladi
```

Kichik batch:

```
kam GPU memory talab qiladi
training noiser bo‘lishi mumkin
```

Agar GPU memory yetmasa:

```
batch=8
```

yoki:

```
batch=4
```

qilib ko‘riladi.

26. Image size nima uchun muhim?

`imgsz` kichik bo‘lsa:

```
tezroq training
kam memory
kichik obyektlar yo‘qolib qolishi mumkin
```

`imgsz` katta bo‘lsa:

```
kichik obyektlar yaxshiroq ko‘rinadi
sekinroq training
ko‘proq GPU memory
```

Misol:

```
imgsz=640
```

ko‘p holatda yaxshi boshlang‘ich.

Kichik obyektlar uchun:

```
imgsz=960
```

yoki:

```
imgsz=1280
```

sinab ko‘riladi.

27. YOLO train qilinganda qayerga saqlanadi?

Ultralytics odatda natijalarni shunday papkaga saqlaydi:

```
runs/
  detect/
    train/
      weights/
```

```
        best.pt
        last.pt
```

Muhim fayllar:

Fayl	Ma’nosi
`best.pt`	validation bo‘yicha eng yaxshi model
`last.pt`	oxirgi epoch modeli

Odatda inference uchun `best.pt` ishlatiladi:

```
model = YOLO("runs/detect/train/weights/best.pt")
```

28. YOLO prediction / inference

Trainingdan keyin modelni yangi rasmda sinaysiz:

```
from ultralytics import YOLO

model = YOLO("runs/detect/train/weights/best.pt")

results = model.predict(
    source="test.jpg",
    conf=0.25,
    save=True
)
```

YOLO prediction mode trained modelni checkpointdan yuklab, yangi image yoki videolarda class va location prediction qilish uchun ishlatiladi.

29. Natijani ko‘rish

```
result = results[0]

boxes = result.boxes

for box in boxes:
    print(box.cls)
    print(box.conf)
    print(box.xyxy)
```

Bu yerda:

```
box.cls  → class id
box.conf → confidence
box.xyxy → x1, y1, x2, y2
```

Ultralytics Results obyektida boxes, masks, keypoints, probabilities va OBB kabi prediction natijalarini ishlatish mumkinligi API hujjatlarida ko‘rsatilgan.

30. Rasmga box chizib ko‘rsatish

```
from ultralytics import YOLO
import matplotlib.pyplot as plt

model = YOLO("runs/detect/train/weights/best.pt")

results = model("test.jpg")

annotated = results[0].plot()

plt.imshow(annotated)
```

```
plt.axis("off")
plt.show()
```

Ultralytics predict hujjatlarida `plot()` metodi bounding box, mask, keypoint va probability kabi predictionlarni original image ustiga chizib berishi aytilgan.

31. Video yoki webcam bilan ishlatish

Video:

```
from ultralytics import YOLO

model = YOLO("best.pt")

results = model.predict(
    source="video.mp4",
    conf=0.25,
    save=True
)
```

Webcam:

```
from ultralytics import YOLO

model = YOLO("best.pt")

results = model.predict(
    source=0,
    conf=0.25,
    show=True
)
```

`source=0` odatda kompyuterning default webcam’ini bildiradi.

32. YOLO tracking

Detection faqat har frame’da obyektni topadi. Tracking esa obyektga ID beradi.

Masalan:

```
person ID 1
person ID 2
car ID 3
```

Bu videoda muhim. Masalan, odam kirib chiqishini sanash, mashina harakatini kuzatish, conveyor belt’da product sanash.

Kod:

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

results = model.track(
    source="video.mp4",
    conf=0.25,
    save=True
)
```

Ultralytics task hujjatlarida tracking YOLO qo‘llaydigan vision tasklardan biri sifatida keltirilgan.

33. YOLO segmentation

Agar box yetarli bo‘lmasa, segmentation kerak bo‘ladi.

Detection:

```
obyekt atrofida box
```

Segmentation:

```
obyektning aniq maskasi
```

Masalan:

```
from ultralytics import YOLO

model = YOLO("yolo11n-seg.pt")

results = model.predict("image.jpg", save=True)
```

YOLO11 hujjatlarida segmentation uchun `yolo11n-seg.pt`, `yolo11s-seg.pt`, `yolo11m-seg.pt`, `yolo11l-seg.pt`, `yolo11x-seg.pt` variantlari borligi ko‘rsatilgan.

Segmentation kerak bo‘ladigan joylar:

```
medical mask
road crack
product defect area
person silhouette
leaf disease area
```

34. YOLO pose estimation

Pose estimation odam yoki obyektning keypointlarini topadi.

Masalan odam uchun:

```
head
shoulder
```

```
elbow
wrist
hip
knee
ankle
```

Kod:

```
from ultralytics import YOLO

model = YOLO("yolo11n-pose.pt")

results = model.predict("person.jpg", save=True)
```

YOLO11 hujjatlarida pose/keypoints uchun `yolo11n-pose.pt` dan `yolo11x-pose.pt` gacha variantlar borligi keltirilgan.

Ishlatilishi:

```
fitness
sport analysis
fall detection
human activity recognition
gesture analysis
```

35. YOLO classification

YOLO faqat detection emas, image classification ham qiladi.

Classification’da butun rasmga bitta label beriladi:

```
cat
dog
healthy_leaf
diseased_leaf
```

Kod:

```
from ultralytics import YOLO

model = YOLO("yolo11n-cls.pt")

results = model.predict("image.jpg")
```

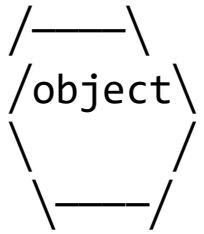
YOLO11 model ro‘yxatida classification uchun `yolo11n-cls.pt`, `yolo11s-cls.pt`, `yolo11m-cls.pt`, `yolo11l-cls.pt`, `yolo11x-cls.pt` variantlari borligi ko‘rsatilgan.

Lekin agar vazifa faqat classification bo‘lsa, ResNet, EfficientNet, ConvNeXt, ViT kabi klassik pretrained classification modellarini ham solishtirish kerak.

36. YOLO OBB — Oriented Bounding Box

Oddiy bounding box doim rasm o‘qlariga parallel bo‘ladi:

OBB esa aylangan box:



Bu ayniqsa shularda foydali:

```
aerial image
ship detection
```

```
text detection
satellite image
rotated objects
warehouse packages
```

YOLO11 hujjatlarida OBB uchun `yolo11n-obb.pt`, `yolo11s-obb.pt`, `yolo11m-obb.pt`, `yolo11l-obb.pt`, `yolo11x-obb.pt` variantlari ko‘rsatilgan.

37. YOLO evaluation metrics

YOLO modelni baholash uchun faqat lossga qaralmaydi. Object detection’da quyidagi metriclar muhim.

37.1. Precision

Precision savoli:

```
Model topgan obyektlarning nechtasi haqiqatan to‘g‘ri?
```

Formula:

```
Precision = TP / (TP + FP)
```

Agar precision past bo‘lsa, model ko‘p noto‘g‘ri obyekt topyapti.

Misol:

```
Model 100 ta helmet topdi.
Ulardan 70 tasi haqiqiy helmet.
Precision = 70%
```

37.2. Recall

Recall savoli:

```
Haqiqiy obyektlarning nechtasini model topa oldi?
```

Formula:

```
Recall = TP / (TP + FN)
```

Agar recall past bo‘lsa, model ko‘p obyektni o‘tkazib yuboryapti.

Misol:

```
Rasmda 100 ta helmet bor.
Model 60 tasini topdi.
Recall = 60%
```

37.3. mAP@50

`mAP@50` — IoU threshold 0.50 bo‘lganda mean Average Precision.

Bu nisbatan yumshoq metric.

37.4. mAP@50-95

`mAP@50-95` — IoU 0.50 dan 0.95 gacha turli thresholdlarda o‘rtacha AP.

Bu qattiqroq metric. Box sifati ham aniqroq baholanadi.

Ultralytics benchmark va validation hujjatlarida model accuracy va speedni baholashda `mAP50-95`, inference time va turli export formatlar bo‘yicha benchmark qilish ishlatilishi aytilgan.

38. Confusion matrix

Object detection’da confusion matrix classlar orasidagi chalkashishni ko‘rsatadi.

Masalan:

```
helmet → no_helmet deb topildi
no_helmet → helmet deb topildi
person → background bo‘lib ketdi
```

Bu model qayerda xato qilayotganini tushunishga yordam beradi.

Agar sizda classlar o‘xshash bo‘lsa:

```
ripe apple / unripe apple
helmet / cap
crack / shadow
healthy / diseased
```

confusion matrix juda muhim.

39. YOLO trainingda losslar

YOLO training paytida model bir nechta narsani o‘rganadi:

```
box joylashuvi
class
object confidence yoki class confidence
```

Turli YOLO versiyalarida loss formulalari farq qiladi, lekin umumiy ma’no shunday:

Loss	Ma’nosi
Box loss	Box koordinatalari qanchalik to‘g‘ri

Class loss	Class qanchalik to‘g‘ri
Objectness/confidence loss	Obyekt bor-yo‘qligi yoki confidence
DFL / regression loss	Ba’zi versiyalarda box regressionni yaxshilash

Yangi YOLO oilalarida loss va head dizayni o‘zgarib boradi. Masalan, YOLO26 docs’da DFL-free regression va end-to-end NMS-free inference muhim yangiliklar sifatida berilgan.

40. Overfitting nima?

Overfitting — model train datasetni yodlab olib, yangi rasmda yomon ishlashi.

Belgilari:

```
train loss yaxshi tushadi
train mAP yuqori
val mAP past yoki oshmaydi
yangi rasmda xato ko‘p
```

Sabablar:

```
dataset kichik
annotation sifati yomon
class imbalance
augmentation kam
epoch juda ko‘p
model juda katta
train/val split noto‘g‘ri
```

Yechimlar:

```
ko‘proq data
yaxshi augmentation
kichikroq model
```

```
early stopping
val setni to‘g‘ri ajratish
label xatolarini tuzatish
```

41. Data leakage

Data leakage — model validation/testda ko‘rmasligi kerak bo‘lgan ma’lumotni bilib qolishi.

Computer vision’da bu ko‘p uchraydi.

Misol:

```
video frame 1 train
video frame 2 val
video frame 3 test
```

Bu xato. Chunki frame’lar deyarli bir xil bo‘lishi mumkin.

To‘g‘ri:

```
butun video train yoki val yoki testga tushsin
bir xil obyekt/odam/product guruh bo‘yicha ajratilsin
```

Aks holda validation natija juda yaxshi ko‘rinadi, lekin real hayotda model yomon ishlaydi.

42. Class imbalance

Masalan:

```
helmet: 5000 image
no_helmet: 300 image
```

Bu holda model helmetni yaxshi o‘rganib, no_helmetni yomon topishi mumkin.

Yechimlar:

```
kam class uchun ko‘proq rasm yig‘ish
augmentation
class balance tekshirish
threshold tuning
confusion matrix analiz
hard example mining
```

Detection’da imbalance faqat image soni emas, obyekt soni bilan ham bog‘liq. Bitta rasmda 20 ta helmet, lekin 1 ta no_helmet bo‘lishi mumkin.

43. Small object muammosi

Kichik obyektlar YOLO’da qiyinroq.

Sabablar:

```
resize paytida obyekt juda kichrayadi
feature mapda signal zaif bo‘ladi
annotation xatosi katta ta’sir qiladi
background bilan aralashib ketadi
```

Yechimlar:

```
imgsz oshirish
slicing / tiling ishlatish
kichik obyektlarni ko‘proq label qilish
yuqori resolution rasm ishlatish
```

```
model size oshirish
augmentationni ehtiyotkor ishlatish
```

Masalan, drone rasmida odam juda kichik bo‘lsa, butun rasmni 640 ga siqish odamni deyarli yo‘qotadi. Bunda rasmni bo‘laklarga bo‘lib detection qilish foydali bo‘lishi mumkin.

44. YOLO uchun pretrained model nima beradi?

Pretrained model odatda COCO kabi katta datasetda o‘qitilgan bo‘ladi. COCO dataset object detection va segmentation uchun keng ishlatiladi va Ultralytics dataset hujjatida 80 object category bilan bog‘liq yirik dataset sifatida keltirilgan.

Pretrained model oldindan shularni o‘rgangan bo‘ladi:

```
edge
texture
object shape
general visual features
common objects
```

Siz esa o‘z classlaringizga moslab fine-tune qilasiz.

Masalan:

```
COCO pretrained YOLO → helmet/no_helmet custom dataset
```

Bu 0 dan train qilishdan ancha yaxshi boshlanish nuqtasi bo‘ladi.

45. Custom YOLO training workflow

Professional workflow shunday:

```
1. Problem definition
2. Class list tayyorlash
3. Image yig‘ish
4. Annotation qilish
5. Datasetni train/val/testga ajratish
6. data.yaml yozish
7. Baseline model train qilish
8. Validation metriclarni ko‘rish
9. Xato rasmlarni analiz qilish
10. Dataset/labelni tuzatish
11. Katta model yoki katta imgsz sinash
12. Final test evaluation
13. Export
14. Deployment
15. Monitoring
```

Bu ketma-ketlikni buzmaslik kerak. YOLO’da kod yozish oson, lekin yaxshi model olish asosan dataset sifati va evaluationga bog‘liq.

46. Amaliy misol: helmet detection

Tasavvur qilaylik sizda 2 class bor:

```
helmet
no_helmet
```

`data.yaml`:

```
path: data/helmet
train: images/train
val: images/val
test: images/test

names:
  0: helmet
  1: no_helmet
```

Training:

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

results = model.train(
    data="data/helmet/data.yaml",
    epochs=100,
    imgsz=640,
    batch=16,
    project="runs/helmet",
    name="yolo11n_baseline"
)
```

Validation:

```
from ultralytics import YOLO

model =
YOLO("runs/helmet/yolo11n_baseline/weights/best.pt"
)

metrics = model.val(
    data="data/helmet/data.yaml",
```

```
    imgsz=640
)
```

Prediction:

```
from ultralytics import YOLO

model =
YOLO("runs/helmet/yolo11n_baseline/weights/best.pt"
)

results = model.predict(
    source="test_images/",
    conf=0.25,
    save=True
)
```

47. CLI bilan ishlatish

Python emas, terminalda ham ishlatish mumkin.

Install:

```
pip install -U ultralytics
```

Ultralytics start page’da package’ni `pip install -U ultralytics` orqali o‘rnatish ko‘rsatilgan.

Train:

```
yolo detect train model=yolo11n.pt data=data.yaml
epochs=100 imgsz=640 batch=16
```

Predict:

```
yolo detect predict
model=runs/detect/train/weights/best.pt
source=test.jpg conf=0.25
```

Val:

```
yolo detect val
model=runs/detect/train/weights/best.pt
data=data.yaml imgsz=640
```

Export:

```
yolo export model=runs/detect/train/weights/best.pt
format=onnx
```

48. YOLO export

Training qilingan model odatda `.pt` formatda bo‘ladi:

```
best.pt
```

Lekin production’da ko‘pincha boshqa format kerak bo‘ladi:

Format	Qayerda ishlatiladi
ONNX	universal inference, ONNX Runtime
TensorRT	NVIDIA GPU’da juda tez
OpenVINO	Intel CPU/GPU
CoreML	iOS/macOS
TFLite / LiteRT	mobile/edge
TorchScript	PyTorch deployment

Ultralytics YOLO26 export hujjatida ONNX, TensorRT va CoreML kabi formatlarga export qilish imkoniyatlari keltirilgan.

Python export:

```
from ultralytics import YOLO
```

```
model = YOLO("best.pt")

model.export(format="onnx")
```

TensorRT:

```
model.export(format="engine")
```

CoreML:

```
model.export(format="coreml")
```

49. Deployment uchun nimani hisoblash kerak?

Production’da faqat accuracy yetmaydi.

Ko‘rish kerak:

```
inference time
FPS
model size
RAM usage
VRAM usage
CPU/GPU load
batch inference
latency
confidence threshold
false positive rate
false negative rate
```

Masalan, factory line’da conveyor belt tez harakat qilsa:

```
model 5 FPS ishlasa yetmasligi mumkin
30 FPS kerak bo‘lishi mumkin
```

Security camera’da esa:

```
5–10 FPS ham ba’zan yetadi
```

50. YOLO va real-time FPS

FPS:

```
frames per second
```

Agar model 30 FPS qilsa, sekundiga 30 frame ishlaydi.

FPS quyidagilarga bog‘liq:

```
model size
image size
hardware
export format
batch size
preprocessing
postprocessing
camera resolution
```

YOLO `n` model tezroq, `x` model sekinroq bo‘ladi.

Agar tezlik kerak bo‘lsa:

```
n yoki s model
imgsz kamaytirish
TensorRT / ONNX ishlatish
FP16
```

```
batch=1
tracking optimize
```

Agar accuracy kerak bo‘lsa:

```
m/l/x model
imgsz oshirish
yaxshi dataset
ko‘proq epoch
augmentation tuning
```

51. YOLO’da hyperparameterlar

Ko‘p ishlatiladigan parametrlar:

Parametr	Ma’nosi
`epochs`	training necha marta datasetni ko‘radi
`imgsz`	input image size
`batch`	batch size
`lr0`	boshlang‘ich learning rate
`patience`	early stopping
`optimizer`	SGD, AdamW va boshqalar
`device`	CPU/GPU tanlash
`workers`	data loading threadlar
`project/name`	natija papkasi
`conf`	inference confidence threshold
`iou`	NMS IoU threshold

Ultralytics config hujjatlarida training settings model performance, speed va accuracyga ta’sir qilishi, batch size, learning rate, momentum va weight decay kabi parametrlar muhimligi aytilgan.

52. YOLO’da augmentation

Augmentation — training paytida rasmni sun’iy o‘zgartirish.

Masalan:

```
flip
scale
crop
rotation
color change
mosaic
mixup
blur
noise
HSV
```

Foydasi:

```
model generalization yaxshilanadi
overfitting kamayadi
turli holatlarga chidamli bo‘ladi
```

Lekin augmentation noto‘g‘ri bo‘lsa, zarar qiladi.

Masalan, text detection’da rasmni aylantirish class ma’nosini o‘zgartirishi mumkin. Medical image’da flip anatomik ma’noni buzishi mumkin. Defect detection’da juda kuchli blur defectni yo‘qotishi mumkin.

53. YOLO natijasi yomon chiqsa nima qilish kerak?

Eng to‘g‘ri debugging tartibi:

53.1. Datasetni ko‘ring

Trainingdan oldin 100 ta random image + labelni ko‘ring.

Savollar:

```
boxlar to‘g‘rimi?
classlar to‘g‘rimi?
hamma obyekt label qilinganmi?
train va val alohidami?
```

53.2. Class distributionni tekshiring

```
class 0: 5000 objects
class 1: 200 objects
```

Agar imbalance bo‘lsa, natijani class-class ko‘rish kerak.

53.3. Confusion matrixni ko‘ring

Qaysi class qaysi class bilan adashyapti?

53.4. False positive rasmlarni ko‘ring

Model yo‘q obyektni topyaptimi?

53.5. False negative rasmlarni ko‘ring

Model haqiqiy obyektni o‘tkazib yuboryaptimi?

53.6. Image size oshiring

Kichik obyektlar bo‘lsa:

```
imgsz=960
```

sinab ko‘riladi.

53.7. Model size oshiring

`n` ishlamasa:

`s → m → l`

sinab ko‘riladi.

53.8. Ko‘proq data qo‘shing

YOLO’da eng katta improvement ko‘pincha modeldan emas, datasetdan keladi.

54. YOLO bilan ishlaganda eng katta 15 xato

1. Direct image emas, noto‘g‘ri file ishlatish

Siz oldin rasm link masalasida ko‘rdingiz: fayl nomi `.jpg` bo‘lsa ham, ichida haqiqiy image bo‘lmasligi mumkin. YOLO’da ham datasetdagi rasm fayllari haqiqiy image bo‘lishi kerak.

2. Label fayl nomi image bilan mos emas

```
image: abc.jpg
label: abc.txt
```

bo‘lishi kerak.

3. Koordinatalar normalized emas

YOLO format:

`0 dan 1 gacha`

Agar pixel koordinata yozib qo‘yilsa, training buziladi.

4. `x_center` o‘rniga `x1` yozish

YOLO label formatda:

```
class x_center y_center width height
```

Agar:

```
class x1 y1 x2 y2
```

berib qo‘yilsa, noto‘g‘ri bo‘ladi.

5. Class index noto‘g‘ri

Labelda `2` bor, lekin `data.yaml`da faqat 2 class:

```
0, 1
```

bo‘lsa, xato.

6. Empty label fayllar bilan chalkashish

Agar rasmda obyekt yo‘q bo‘lsa, label fayl bo‘sh bo‘lishi mumkin. Lekin bu holatni to‘g‘ri boshqarish kerak.

7. Train/val split yomon

Bir xil scene train va valga tushsa, metric ishonchsiz bo‘ladi.

8. Juda oz rasm

10–20 rasm bilan yaxshi model kutish noto‘g‘ri.

9. Annotation sifati past

Box yomon bo‘lsa, model ham yomon.

10. Classlar aniq ta’riflanmagan

Masalan:

```
small crack
large crack
scratch
defect
```

class farqlari annotatorga ham tushunarli bo‘lishi kerak.

11. Faqat accuracyga qarash

Detection’da mAP, precision, recall, false negative/positive muhim.

12. Threshold noto‘g‘ri

`conf=0.8` qilib qo‘ysangiz ko‘p obyekt yo‘qolishi mumkin.

13. Kichik obyektlarda image size kichik

`imgsz=640` ba’zan yetmaydi.

14. Katta modelni kuchsiz device’da ishlatish

`x` model accuracy yaxshi bo‘lishi mumkin, lekin real-time ishlamasligi mumkin.

15. Deploymentni oxirida o‘ylash

Loyiha boshida target device aniq bo‘lishi kerak:

```
CPU?
GPU?
mobile?
```

```
webcam?
server?
```

55. YOLO loyihasi uchun professional folder structure

```
yolo_project/
  data/
    raw/
    processed/
    images/
      train/
      val/
      test/
    labels/
      train/
      val/
      test/
    data.yaml

  notebooks/
    01_check_images.ipynb
    02_visualize_labels.ipynb
    03_train_yolo.ipynb
    04_evaluate_errors.ipynb

  src/
    check_dataset.py
    train.py
    predict.py
    export.py
```

```
    visualize.py

  runs/
    detect/

  models/
    best.pt
    best.onnx

  reports/
    confusion_matrix.png
    results.csv
    sample_predictions/

  README.md
  requirements.txt
```

Bu struktura GitHub uchun ham yaxshi.

56. Datasetni tekshiradigan foydali kod

```
from pathlib import Path
from PIL import Image

image_dir = Path("data/images/train")
label_dir = Path("data/labels/train")

image_paths = list(image_dir.glob("*.jpg")) +
list(image_dir.glob("*.png")) +
list(image_dir.glob("*.jpeg"))

print("Image count:", len(image_paths))
```

```
missing_labels = []

for img_path in image_paths:
    label_path = label_dir / f"{img_path.stem}.txt"

    if not label_path.exists():
        missing_labels.append(img_path.name)
        continue

    try:
        img = Image.open(img_path)
        w, h = img.size
    except Exception as e:
        print("Broken image:", img_path, e)
        continue

    lines =
label_path.read_text().strip().splitlines()

    for i, line in enumerate(lines):
        parts = line.split()

        if len(parts) != 5:
            print("Wrong label format:",
label_path, "line", i + 1, line)
            continue

        cls, x, y, bw, bh = parts
        x, y, bw, bh = map(float, [x, y, bw, bh])

        if not (0 <= x <= 1 and 0 <= y <= 1 and 0
<= bw <= 1 and 0 <= bh <= 1):
            print("Not normalized:", label_path,
"line", i + 1, line)
```

```
print("Missing labels:", len(missing_labels))
print(missing_labels[:10])
```

Bu kod quyidagilarni tekshiradi:

```
image bor-yoʻqligi
label bor-yoʻqligi
label 5 ustundan iboratmi
koordinatalar 0-1 oraligʻidami
image buzilganmi
```

57. Predictionlarni DataFrame qilib olish

```
from ultralytics import YOLO
import pandas as pd

model = YOLO("best.pt")

results = model.predict("test.jpg", conf=0.25)

rows = []

for r in results:
    for box in r.boxes:
        cls_id = int(box.cls[0])
        conf = float(box.conf[0])
        x1, y1, x2, y2 = box.xyxy[0].tolist()

        rows.append({
            "class_id": cls_id,
            "class_name": model.names[cls_id],
            "confidence": conf,
```

```
            "x1": x1,
            "y1": y1,
            "x2": x2,
            "y2": y2
        })

df = pd.DataFrame(rows)
print(df)
```

Bu production yoki analysis uchun juda foydali.

58. YOLO bilan object counting

Masalan, rasmda nechta odam borligini sanash:

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

results = model.predict("street.jpg", conf=0.25)

person_count = 0

for box in results[0].boxes:
    cls_id = int(box.cls[0])
    class_name = model.names[cls_id]

    if class_name == "person":
        person_count += 1

print("Person count:", person_count)
```

Buni quyidagilarga qo‘llash mumkin:

```
people counting
car counting
product counting
fruit counting
animal counting
```

59. YOLO va CCTV / video analytics

YOLO video analytics’da ko‘p ishlatiladi.

Masalan:

```
line crossing
zone intrusion
vehicle counting
helmet violation
mask detection
fall detection
fire/smoke detection
```

Bunda detection + tracking + logic ishlatiladi.

Pipeline:

```
video frame
   ↓
YOLO detection
   ↓
tracker
   ↓
object ID
   ↓
business rule
```

```
   ↓
alert / count / log
```

Masalan, odam xavfli zonaga kirsa:

```
person box zone ichida bo‘lsa → alert
```

60. YOLO va industrial defect detection

YOLO sanoatda ham ishlatiladi:

```
scratch
crack
missing part
wrong label
broken product
foreign object
package damage
```

Lekin defect detection’da muammo ko‘p:

```
defectlar kichik
dataset kam
class imbalance kuchli
lighting o‘zgaradi
false positive qimmatga tushadi
```

Bunda YOLO bilan birga anomaly detection yoki segmentation model ham sinab ko‘riladi.

61. YOLO va medical image

Tibbiy rasmlarda YOLO ishlatilishi mumkin:

```
lesion detection
tumor localization
cell detection
pill detection
fracture region
```

Lekin ehtiyot bo‘lish kerak:

```
expert annotation kerak
metriclar qat’iy bo‘lishi kerak
false negative xavfli
data privacy muhim
clinical validation kerak
```

Medical tasklarda YOLO faqat texnik model emas, balki validatsiya va mas’uliyat masalasi ham bor.

62. YOLO va agriculture

Agriculture’da YOLO ko‘p ishlatiladi:

```
fruit detection
leaf disease
weed detection
pest detection
crop counting
animal monitoring
```

Agriculture bo‘yicha YOLO ishlatilishiga oid review ishlarida YOLO real-time monitoring, surveillance, sensing, automation va robotics kabi vazifalarda qo‘llanayotgani qayd etilgan.

63. YOLO va OCR / text detection

YOLO text detection uchun ham ishlatilishi mumkin:

```
license plate detection
document text area
shop sign detection
container number
product label area
```

Lekin YOLO odatda textni o‘qimaydi, faqat text joyini topadi. Textni o‘qish uchun OCR model kerak:

```
YOLO → text region detection
OCR → text recognition
```

Masalan:

```
YOLO plate box topadi
OCR plate raqamini o‘qiydi
```

64. YOLO va segmentation farqi

Detection:

```
box bilan topadi
```

Segmentation:

```
mask bilan aniq ajratadi
```

Misol:

```
Detection: olma atrofida to‘rtburchak
Segmentation: olmaning aniq shakli
```

Agar hisoblashda obyekt maydoni kerak bo‘lsa, segmentation yaxshi.

Masalan:

```
leaf disease area percentage
crack area
person silhouette
road damage area
```

65. YOLO training natijasini qanday o‘qish kerak?

Trainingdan keyin ko‘pincha grafiklar chiqadi:

```
train/box_loss
train/cls_loss
val/box_loss
val/cls_loss
metrics/precision
metrics/recall
metrics/mAP50
metrics/mAP50-95
```

Yaxshi holat:

```
loss tushadi
precision oshadi
recall oshadi
mAP oshadi
val metric train bilan juda katta farq qilmaydi
```

Yomon holat:

```
train yaxshi, val yomon → overfitting
precision yuqori, recall past → model kam topyapti
recall yuqori, precision past → model ko‘p
noto‘g‘ri topyapti
mAP50 yaxshi, mAP50-95 past → boxlar aniq emas
```

66. Precision va recall balansini qanday tanlash kerak?

Bu taskga bog‘liq.

No_helmet detection

No_helmetni o‘tkazib yubormaslik muhim:

```
recall muhim
```

Defect detection

Defectni o‘tkazib yuborish qimmat:

```
recall muhim
```

Automatic rejection system

Yaxshi mahsulotni noto‘g‘ri reject qilish zarar:

```
precision ham juda muhim
```

Human review bor bo‘lsa

Model ko‘proq candidate bersa ham odam tekshirsa:

```
recallni oshirish mumkin
```

To‘liq avtomatik system

False positive ham, false negative ham qimmat:

`thresholdni ehtiyotkor tuning qilish kerak`

67. YOLO’da confidence threshold tuning

Inference:

```
model.predict("image.jpg", conf=0.25)
```

`conf=0.25` — ko‘p boshlang‘ich holatda ishlatiladi.

Agar false positive ko‘p bo‘lsa:

`conf=0.5`

Agar obyektlar topilmay qolsa:

`conf=0.15`

Lekin thresholdni taxmin bilan emas, validation setda sinab tanlash kerak.

68. YOLO’da IoU threshold tuning

NMS ishlatiladigan variantlarda `iou` threshold bir-biriga o‘xshash boxlarni qanday olib tashlashni boshqaradi.

```
model.predict("image.jpg", conf=0.25, iou=0.45)
```

Agar bir obyektga ko‘p box chiqsa:

`iou thresholdni moslash kerak`

Agar yaqin turgan obyektlar bitta bo‘lib qolsa:

```
NMS juda agressiv bo‘lishi mumkin
```

69. YOLO va hardware

YOLO quyidagi joylarda ishlashi mumkin:

```
CPU
NVIDIA GPU
Jetson
Raspberry Pi
mobile
server
cloud
browser orqali ba’zi formatlarda
```

Lekin model tanlash hardwarega bog‘liq.

CPU

```
n/s model
ONNX/OpenVINO
kichik imgsz
```

NVIDIA GPU

```
m/l/x ham mumkin
TensorRT yaxshi
FP16 foydali
```

Jetson

```
n/s model
TensorRT
imgsz optimizatsiya
```

Mobile

```
n model
CoreML/TFLite/LiteRT
quantization
```

70. YOLO exportdan keyin natija farq qilishi mumkin

PyTorch `.pt` model bilan ONNX/TensorRT natija ozgina farq qilishi mumkin. Sabab:

```
operator conversion
precision FP32/FP16/INT8
NMS implementation
preprocessing farqi
postprocessing farqi
```

Shuning uchun exportdan keyin albatta test qilish kerak:

```
same images
same confidence
same input size
same preprocessing
same metric
```

71. YOLO’da reproducibility

Agar har safar training natijasi farq qilsa, bu normal bo‘lishi mumkin. Sabab:

```
random initialization
augmentation randomness
GPU nondeterminism
data loading order
```

Qisman nazorat qilish uchun seed qo‘yiladi:

```
results = model.train(
    data="data.yaml",
    epochs=100,
    imgsz=640,
    seed=42
)
```

Lekin deep learning’da 100% bir xil natija har doim kafolatlanmaydi.

72. YOLO bilan yaxshi loyiha qilish uchun minimal reja

Agar siz hozir YOLO o‘rganmoqchi bo‘lsangiz, shu ketma-ketlik eng yaxshi:

```
1. Pretrained YOLO bilan oddiy image prediction
2. Video/webcam prediction
3. COCO classlarini tushunish
4. Custom dataset formatini o‘rganish
5. 2 classli kichik dataset annotation qilish
```

6. YOLO train qilish
7. results.png va confusion matrix o‘qish
8. best.pt bilan prediction
9. false positive / false negative analiz
10. ONNX export
11. real-time demo

73. YOLO bilan birinchi amaliy kod

```
from ultralytics import YOLO

# Pretrained model yuklash
model = YOLO("yolo11n.pt")

# Rasmda obyektlarni topish
results = model.predict(
    source="image.jpg",
    conf=0.25,
    save=True
)

# Natijalarni chiqarish
for result in results:
    for box in result.boxes:
        class_id = int(box.cls[0])
        class_name = model.names[class_id]
        confidence = float(box.conf[0])
        coordinates = box.xyxy[0].tolist()

        print(class_name, confidence, coordinates)
```

Bu kod:

```
1. pretrained YOLO yuklaydi
2. image.jpg ni tekshiradi
3. topilgan obyektlarni chiqaradi
4. box chizilgan rasmni saqlaydi
```

74. Custom dataset bilan birinchi training kodi

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

results = model.train(
    data="data.yaml",
    epochs=50,
    imgsz=640,
    batch=8,
    project="runs/custom",
    name="first_yolo"
)
```

Trainingdan keyin:

```
best_model =
YOLO("runs/custom/first_yolo/weights/best.pt")

best_model.predict(
    source="test_images",
    conf=0.25,
    save=True
)
```

75. Google Colab’da YOLO ishlatish

Siz Colab T4 ishlatgansiz. YOLO uchun Colab yaxshi.

Install:

```
!pip install -U ultralytics
```

GPU tekshirish:

```
import torch
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0))
```

Train:

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

results = model.train(
    data="/content/dataset/data.yaml",
    epochs=100,
    imgsz=640,
    batch=16,
    device=0
)
```

Agar CUDA memory error chiqsa:

```
batch=8
```

yoki:

```
batch=4
```

qiling.

76. YOLO output papkasini Google Drive’ga saqlash

Colab’da runtime o‘chsa, fayllar yo‘qoladi. Drive mount qilish kerak:

```
from google.colab import drive
drive.mount('/content/drive')
```

Project path:

```
results = model.train(
    data="/content/dataset/data.yaml",
    epochs=100,
    imgsz=640,
    batch=16,
    project="/content/drive/MyDrive/yolo_runs",
    name="experiment_1"
)
```

Shunda `best.pt` Drive’da qoladi.

77. YOLO va OpenCV

YOLO natijasini OpenCV bilan ishlatish mumkin.

Masalan webcam:

```
import cv2
from ultralytics import YOLO

model = YOLO("best.pt")
```

```
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    if not ret:
        break

    results = model.predict(frame, conf=0.25,
verbose=False)

    annotated = results[0].plot()

    cv2.imshow("YOLO", annotated)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

Bu real-time demo uchun yaxshi.

78. YOLO modelni qanday tanlash kerak?

Boshlash uchun

```
yolo11n.pt yoki yolo11s.pt
```

GPU kam bo‘lsa

```
n
```

Accuracy kerak bo‘lsa

```
m yoki l
```

Production real-time bo‘lsa

```
n/s + TensorRT/ONNX
```

Kichik obyektlar bo‘lsa

```
imgsz oshirish
m/l model sinash
slicing
```

Segmentation kerak bo‘lsa

```
-seg model
```

Pose kerak bo‘lsa

```
-pose model
```

Rotated object bo‘lsa

```
-obb model
```

79. YOLO va boshqa detectorlar solishtirish

Model	Kuchli tomoni	Kamchiligi
YOLO	tez, amaliy, deployment oson	kichik obyektlarda tuning kerak
Faster R-CNN	accuracy yaxshi bo‘lishi mumkin	sekinroq
RetinaNet	imbalancega yaxshi	YOLO kabi sodda emas
DETR	transformer, end-to-end g‘oya	training/compute murakkabroq

RT-DETR	real-time transformer detection	deployment va ecosystem farq qiladi
SSD	yengil klassik detector	zamonaviy YOLOlardan past bo‘lishi mumkin

Amaliyotda custom real-time detection uchun YOLO juda qulay boshlang‘ich nuqta.

80. YOLO haqida eng muhim xulosa

YOLO — object detection uchun eng amaliy va tez ishlatiladigan deep learning model oilalaridan biri. Uning asosiy g‘oyasi: rasmga bir marta qarab, obyektlarning classi va joylashuvini birdan taxmin qilish. Original YOLO detection’ni bitta regression problem sifatida ko‘rsatgani bilan mashhur bo‘lgan.

Amaliy loyihada YOLO’dan to‘g‘ri foydalanish uchun eng muhim narsalar:

```
1. Dataset sifati
2. Annotation sifati
3. To‘g‘ri train/val/test split
4. To‘g‘ri model size tanlash
5. imgsz va batchni hardwarega moslash
6. mAP, precision, recallni to‘g‘ri o‘qish
7. False positive / false negative analiz
8. Deployment formatini oldindan o‘ylash
```

YOLO’da kuchli model olish faqat `model.train()` yozish bilan tugamaydi. Eng katta ish — **datasetni to‘g‘ri tayyorlash, labelni tekshirish, xatolarni ko‘rish, metriclarni tushunish va real-world test qilish**. Shular to‘g‘ri bo‘lsa, YOLO bilan juda kuchli computer vision loyihalar qilish mumkin.